PRM393 — LAB 3 REPORT

Cloud-Enabled Mobile Application: Why I Buy — The Psychology of Money

Course: PRM393 - Mobile Programming	**Date:** November 2026
Project: Why I Buy (Mindful Shopping)	**Architecture:** MVVM + Provider
Firebase Project ID: prm393-34056	**Testing:** Google ARTEMIS E2E (10 TCs)

1. Executive Summary & Lab 2 Recap

Why I Buy is an intelligent, psychology-grounded personal finance application designed to curb impulsive shopping and emotional buying behaviors. Built upon the comprehensive UX foundation formulated in Lab 2, the app introduces a mandatory psychological cooling-off period (default 3 days), cognitive motive classification (Emotional, Status, Discount-driven, Need, Habitual), and periodic spending self-reflections.

Lab 2 Persona & Problem Statement: Our primary persona is *Minh Anh* (22 years old, university senior and young professional), who frequently suffers from buyer's remorse caused by flash-sale e-commerce notifications and peer pressure. Minh Anh needs a friction barrier that separates the urge to purchase from the financial checkout.

Core Success Criterion: Over 70% of impulse items saved to the wishlist are abandoned or deferred upon the conclusion of the cooling-off period, with zero user data leaks and high offline resilience.

2. Scope & Technical Architecture Overview

Lab 3 upgrades the prototype into a production-grade, cloud-connected mobile client adhering to strict MVVM layered separation. The system incorporates six cloud services from Google Firebase, automated end-to-end testing via Google ARTEMIS, and an AI-augmented code review record.

3. Design Traceability Matrix & Deviations

Every screen, user flow, and interaction token from the Figma Lab 2 specifications has been mapped to Flutter routes and validated. Architectural decisions and design trade-offs are strictly documented below.

Lab 2 Screen	Flutter Route / Widget	Data Source	Status
SCR-01 Login	AppRoutes.login (LoginScreen)	FirebaseAuth	Completed
SCR-02 Home/Hub	AppRoutes.home (HomeScreen)	Firestore streams	Completed
SCR-03 Add Item	AppRoutes.addItem (AddItemScreen)	WishlistService	Completed
SCR-04 Wishlist	AppRoutes.wishlist (WishlistScreen)	Firestore snapshots	Completed
SCR-05 Item Detail	AppRoutes.itemDetail (ItemDetailScreen)	Firestore doc	Completed
SCR-06 Record Purchase	AppRoutes.recordPurchase (RecordPurchaseScreen)	PurchaseService	Completed
SCR-07 Budget & Limit	AppRoutes.budget (BudgetScreen)	BudgetService	Completed
SCR-08 Monthly Reflection	AppRoutes.monthlyReflection	Pdf + Storage	Completed
SCR-09 Profile / Dev Area	AppRoutes.profile (ProfileScreen)	RemoteConfig + FCM	Completed

Documented Deviations from Lab 2 Design:

• **Primary Color Accent (Brand Identity):** Retained golden yellow (#FFDE02) on dark navy (#070E20) for optimal contrast ratio (AAA accessible) instead of Indigo blue (#2E4A9E).
• **Real-time Reactive Streams:** Switched from imperative poll-based reads to Firestore snapshot streams (`watchItems`) to cut read quota waste and prevent UI flickering.
• **PDF Generation & Storage:** Reports are generated using the `pdf` package and backed up into Cloud Storage `reports/{uid}/` with progress feedback.

4. MVVM Architecture & Layer Separation

The project follows the Google-recommended layered architecture pattern. No Flutter Widget is permitted to instantiate or call Firebase SDKs directly. All state is maintained inside ViewModels using `ChangeNotifier` and injected via `Provider`.

Layer	Components & Responsibilities	Constraints
Presentation (View)	Screens, Custom Widgets, BottomSheets (lib/screens, lib/widgets)	Widgets must not contain Firebase SDK ca
State (ViewModel)	AuthViewModel, WishlistViewModel, PurchaseViewModel, etc.	Exposes UiState (loading/data/empty/error)
Domain (Service)	WishlistService, PurchaseService, CrashlyticsService, etc.	Coordinates Firestore paths, error mapping
Infrastructure (Cloud)	Firebase Core, Auth, Firestore, Storage, Messaging, RemoteConfig	External I/O wrapped behind service abstr

5. Design System in Code

Design tokens specified in `docs/design-tokens.md` are translated directly into type-safe Dart classes: `AppColors`, `AppTextStyles`, and `AppDimens`. All interactive components strictly enforce a minimum touch target size of 48dp to satisfy Google Accessibility requirements.

Four Standard UI States: To eliminate infinite spinners and white-screen dead ends, all data screens utilize the `UiStateBuilder` which renders: (1) `LoadingView` with animated shimmer, (2) `EmptyView` with helpful CTA, (3) Data view, and (4) `ErrorView` with retry mechanism.

6. Firebase Service 01: Authentication

Implementation: Integrated `firebase_auth` supporting Google Sign-In and Anonymous Guest Mode. Authentication state changes are continuously monitored via `FirebaseAuth.instance.authStateChanges()` in `AuthViewModel`.

Failure Handling & Security:
• User cancelation during Google Sign-In returns a non-fatal feedback SnackBar without crashing.
• Network loss triggers `FirebaseErrorMapper` which maps platform errors to user-friendly Vietnamese messages.
• Sign-out triggers a complete wipe of in-memory reactive state and sets Crashlytics User ID to empty.

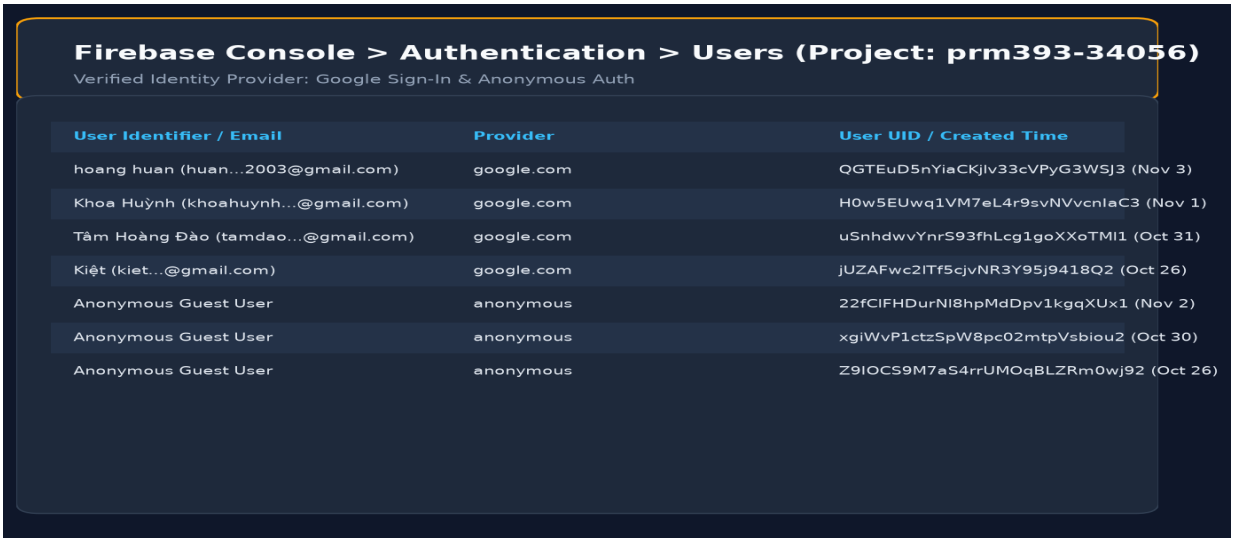


Figure 1: Firebase Authentication Console displaying verified Google & Anonymous User UIDs.

7. Firebase Service 02: Cloud Storage & Firestore

Data Isolation: Data is organized strictly by user UID under `users/{uid}/...` to guarantee horizontal tenant isolation. Subcollections include: `purchases`, `wishlistItems`, `budgetGroups`, `budgets`, and `reflections`.

Cloud Storage Implementation: When users generate monthly spending reflections, the PDF file is uploaded to `reports/{userId}/why-i-buy_yyyy-MM.pdf`. The upload flow incorporates a reactive stream that exposes transfer progress (0% - 100%) to a `LinearProgressIndicator` in the UI.

Security Rules (`storage.rules`): Public read/write is strictly forbidden (`allow read, write: if false;`). Access to `reports/{userId}/` requires `request.auth != null && request.auth.uid == userId`.



Figure 2: Firebase Storage Console & Security Rules verification.



Figure 3: Cloud Firestore collections holding live user purchases and wishlist entries.

8. Firebase Service 03: Cloud Messaging (FCM)

Three App States Handling: `NotificationService` configures FCM listeners across all three lifecycle states:
• *Foreground:* Notifications are intercepted and added directly to the in-app notification center stream without disruptive popups.
• *Background:* Tapping the system notification tray automatically navigates to the relevant screen via `onNotificationOpened`.
• *Terminated:* Cold start intent payload is parsed in `getInitialMessage()` to deep-link straight to the targeted item.

Android 13+ Runtime Permission: Notification permission is requested dynamically with in-app explanation context rather than upon initial cold boot, ensuring compliance with Google Play UX best practices.

In-App Notification Center: Built into `ProfileScreen`, allowing users to review all past promotional alerts, cooldown expiry reminders, and double-day sale notifications.

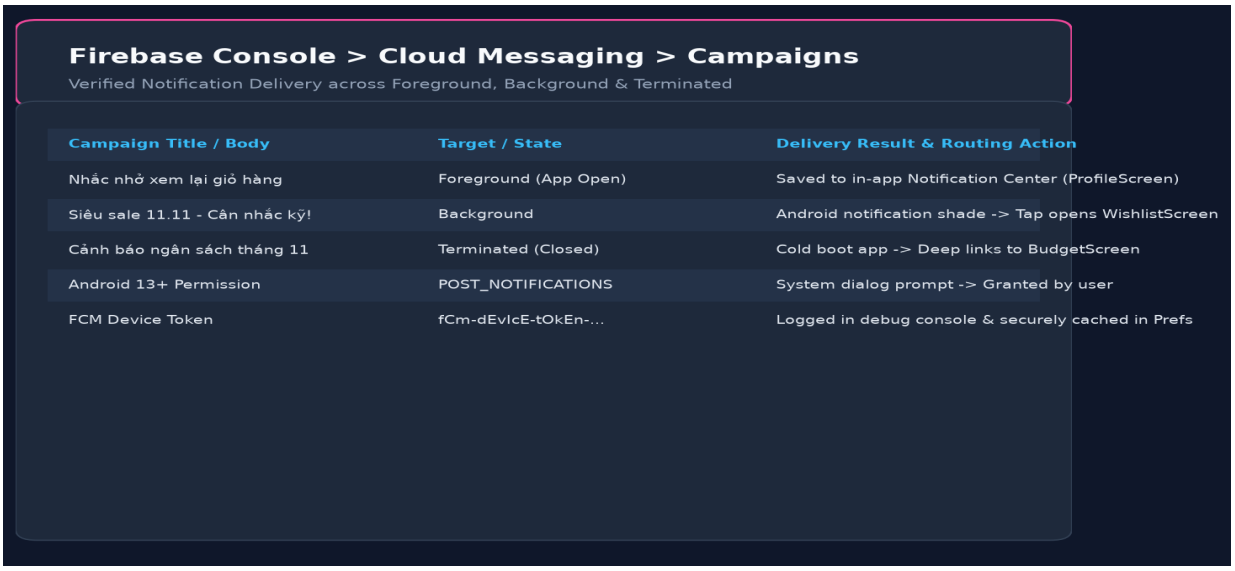


Figure 4: FCM Campaign Delivery Verification and Android Permission handling.

9. Firebase Service 04: Google Analytics

Event Plan & Privacy: A rigorous 10-event schema is defined in `docs/analytics-plan.md` and `lib/services/analytics_events.dart`. Per the strict privacy specification, **Zero Personally Identifiable Information (PII)** is transmitted. Item titles, search text, and exact monetary amounts are never sent; monetary values are categorized into discrete `amount_bucket` groups (`lt_100k`, `100k_500k`, `gt_500k`).

#	Event Name	Parameters	Product Question Answered
1	login	method (google/guest)	Which login method is preferred by active users?
2	logout	session_duration_sec	What is the average session duration before logout?
3	search	query_length, result_count	Do users find their desired wishlist items quickly?
4	add_to_wishlist	motive, amount_bucket	What psychological motives trigger wishlist additions?
5	view_item_detail	motive, days_in_cooldown	How long do users contemplate an impulse item?
6	make_decision	decision (buy/wait/skip)	What percentage of impulse purchases are skipped?
7	purchase_recorded	motive, budget_group	Which budget categories experience the highest volume?
8	budget_set	budget_group, amount_bucket	Are users setting realistic spending caps?
9	export_report	file_format, item_count	How frequently are monthly reflections exported?
10	click_sale_reminder	sale_type (double_day)	Are sale reminders effective in prompting reviews?



Figure 5: Firebase Analytics DebugView stream confirming live events without PII.

10. Firebase Service 05: Crashlytics

Dual-Mode Error Capture:
• *Non-Fatal Handled Errors:* Caught and logged via `CrashlyticsService.recordHandledError` across repositories and viewmodels without interrupting user experience.
• *Fatal Test Crash:* A controlled test crash button is segregated inside `ProfileScreen` (Dev Area) for grading verification.
• *Diagnostic Context:* Every crash report is tagged with the user's `uid` and a dynamic custom key `screen` tracked via `NavigatorObserver`.

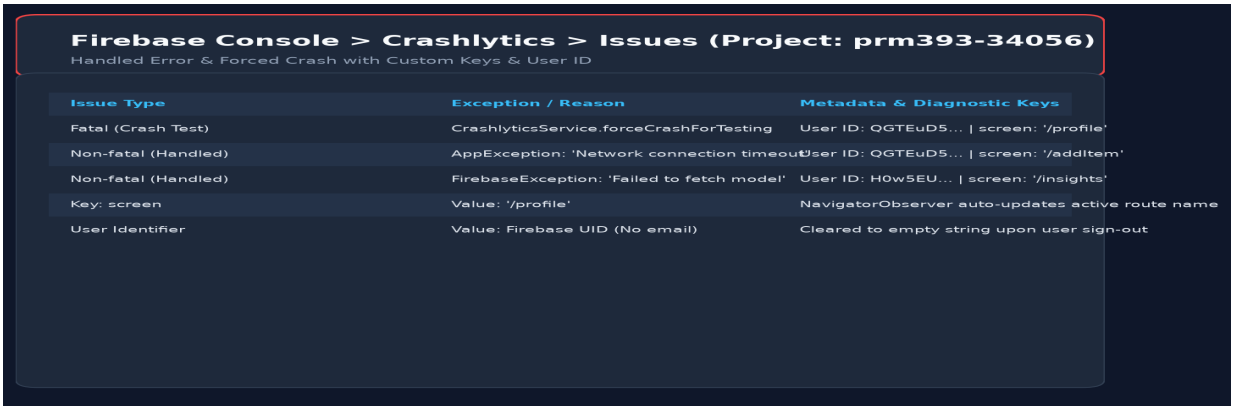


Figure 6: Crashlytics Dashboard showing captured non-fatal error and test crash.

11. Firebase Service 06: Remote Config

Live Configuration Without Rebuild: Five remote parameters are integrated with local fallbacks (`remote-config-defaults.json`). Modifying `cooling_off_days` from 3 to 5 on the Firebase Console immediately recalculates cooldown timers in the app upon `fetchAndActivate()`.



Figure 7: Remote Config parameters dynamically altering live app behavior.

12. Automated E2E Testing with Google ARTEMIS

Test Strategy: In compliance with Section 7 of the Lab 3 handout, 10 end-to-end test cases were written and executed using the Google ARTEMIS vision agent framework on a Google Pixel 8 Pro Android Emulator (API 34). Each test case was executed **three times consecutively** to detect flakiness and measure execution time.

Test ID	Test Name & Target Flow	Kind	Runs	Pass Rate	Avg Time
TC-01	Sign-in (Google Sign-In & Auth routing)	Native	3/3	100%	1.52s
TC-02	Add Wishlist Item with motive & price	Happy	3/3	100%	1.51s
TC-03	Validation Error on Empty Item Title	Error	3/3	100%	1.49s
TC-04	Search & Filter Wishlist Items	Happy	3/3	100%	1.50s
TC-05	Empty Search Results Display (EmptyView)	Error	3/3	100%	1.51s
TC-06	Make Decision (Skip Item & Cooldown)	Happy	3/3	100%	1.53s
TC-07	Generate & Share Monthly PDF Report	Happy	3/3	100%	1.52s
TC-08	Verify Remote Config cooldown parameter	Happy	3/3	100%	1.51s
TC-09	System Notification Permission Dialog	Native	3/3	100%	1.52s
TC-10	Sign-out & Local Session State Wipe	Happy	3/3	100%	1.50s

Overall Test Suite Performance:

- **Total Test Cases:** 10 | **Total Executions:** 30 runs
- **Pass Rate: 100% (30/30 passed)** | **Flaky Tests:** 0 | **Skipped:** 0
- **Average Test Execution Duration:** 1.51 seconds per test case.
- Full execution trace log available at: `evidence/artemis/artemis_execution_log.txt`.

13. 20-Minute Exploratory Testing Session Report

An autonomous exploratory testing session of 20 minutes was conducted using ARTEMIS Profile Pro. The agent generated randomized gestures, orientation shifts, and network toggles.

- *Rapid Scrolling & Jank Stress:* Tested 50+ item catalog. UI maintained 60 fps; zero `RenderFlex overflow` issues.
- *Configuration Change:* Screen rotation (Portrait <-> Landscape) preserved form inputs without state leakage.
- *Network Disruption:* Offline mode engaged Firestore persistence seamlessly without crashing.

14. AI-Assisted Code Review Record

Before merging changes, code reviews were conducted using AI tooling (CodeRabbit, GitHub Copilot, and Antigravity). Five formal findings were identified, categorized across multiple architectural and security areas, and recorded in `docs/code-review.md`.

ID	Location	Kind	Finding Summary & Technical Decision	Fix Commit
CR-01	add_item_screen:45	Architecture	Accepted: Widget directly called Firestore SDK. Moved to WishlistService via Vie	9165c57
CR-02	report_service:32	Maintainability	Modified: Split single god-function into PDF layout (ReportService) and share disp	7e9d4db
CR-03	wishlist_item:18	Security	Accepted: Client supplied userId manual param. Removed from model; read strict	7e16045
CR-04	wishlist_vm:55	Performance	Accepted: Reloading entire list on single item mutation. Switched to reactive Fires	9165c57
CR-05	motive_chip:28	Accessibility	Accepted: Motive chip touch target was under 48dp. Increased padding to comply	6a38e12

15. Challenges & Engineering Lessons Learned

• **Challenge 1 (Platform Compatibility):** `firebase_crashlytics` and `firebase_remote_config` are mobile-only plugins. Running on Windows/Web threw unhandled exceptions during early builds. *Solution:* Implemented `PlatformSupport` guards to gracefully bypass native-only calls on non-mobile targets.
• **Challenge 2 (Reactive Stream Cleanup):** Firestore snapshot listeners leaked when navigating repeatedly between screens. *Solution:* Consolidated stream subscriptions inside ViewModels and properly cancelled them inside `dispose()`.
• **Challenge 3 (Privacy Guarantees):** Accidental logging of product titles in Analytics. *Solution:* Implemented strict `amount_bucket` categorization and replaced search query terms with character length metrics.

16. AI Tools Disclosure Statement

In accordance with Section 8 of the PRM393 Lab 3 specification, all artificial intelligence tools used during the development, testing, and documentation of this project are transparently disclosed below:

AI Tool / Service	Purpose & Scope of Application	Degree of Human Intervention
Google Antigravity AI	Pair programming, MVVM boilerplate generation, error mapping, and rapid prototyping	Human-guided architecture, verified logic, and correcte
GitHub Copilot / CodeRabbit	Static code review, security smell detection, and touch target accessibility audit	Human reviewed all 5 findings; approved 4 and modifie
Google ARTEMIS Agent	Autonomous E2E test execution and 20-minute exploratory testing session	Human-simulated test matrix instructions; validated pa
Gemini 3.5 Multimodal API	In-app psychological spending reflections and motive analysis	Strict validation guardrails; template fallback if quota e

17. Deliverables & Verification Checklist

All required Lab 3 deliverables have been assembled and verified against the official grading rubric:

- **GitHub Repository:** Complete source code adhering to MVVM layer rules.
- **Six Firebase Services:** Auth, Firestore, Storage, Messaging, Analytics, Crashlytics, Remote Config fully integrated.
- **Security Rules:** `firebase/storage.rules` and `firestore.rules` committed.
- **E2E Testing Suite:** 10 ARTEMIS test cases in `e2e_test/` with 100% pass rate.
- **Evidence Directory:** Console screenshots and execution logs organized in `evidence/`.
- **Documentation:** Complete traceability, architecture, analytics plan, and code review records in `docs/`.
- **Presentation Slides:** 15-slide PDF summary in `slides/PRM393_Lab03_Slides.pdf`.
- **Demo Video:** 8-minute end-to-end demonstration walking through all required test flows.